

Київський національний університет імені Тараса Шевченка
Факультет комп'ютерних наук та кібернетики

Звіт

з лабораторної роботи №3
з моделювання складних систем

Варіант 1

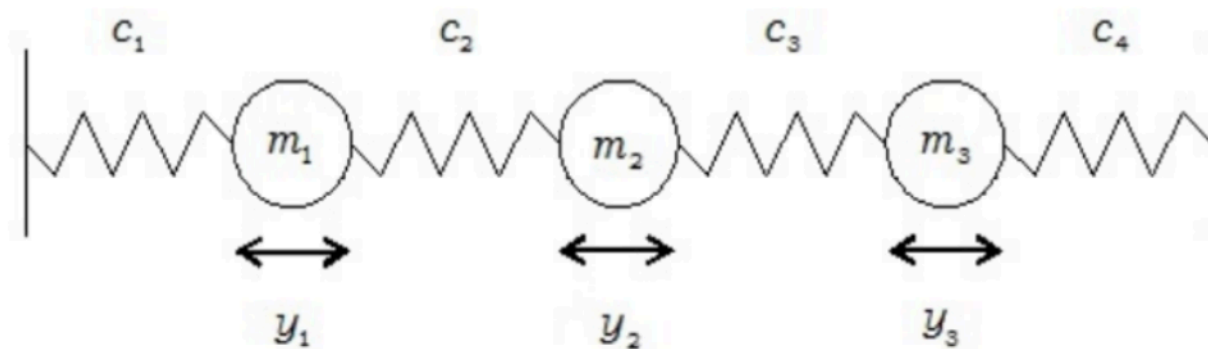
Виконала:
Студентка групи ІПС-32
Клевчук Марія Вячеславівна

Київ
2025

Параметрична ідентифікація параметрів з використанням функцій чутливості.

Постановка задачі

Для математичної моделі коливання трьох мас m_1, m_2, m_3 , які поєднані між собою пружинами з відповідними жорсткостями c_1, c_2, c_3, c_4 , і відомої функції спостереження координат моделі $\bar{y}(t), t \in [t_0, t_k]$ потрібно оцінити частину невідомих параметрів моделі з використанням функції чутливості.



Спостереження стану моделі проведені на інтервалі часу $t_0 = 0, t_k = 50, \Delta t = 0.2$.

За варіантом нам треба знайти наступні значення: $\beta = (c_1, m_1, m_2)^T$

Алгоритм

Метод параметричної ідентифікації з використанням функцій чутливості є ітераційним методом для знаходження параметрів β , які мінімізують розбіжність між розрахунковими та спостережуваними даними.

Наша мета - мінімізувати функціонал якості $I(\beta)$. Це сума квадратів різниць між спостережуваними даними $y(t)$ та розрахунковими даними $y(t, \beta)$ моделі. Чим більше значення, тим більш «неточною» є модель.

Показник якості ідентифікації параметрів β має вигляд

$$I(\beta) = \int_{t_0}^T (\bar{y}(t) - y(t, \beta))^T (\bar{y}(t) - y(t, \beta)) dt \rightarrow \min_{\beta}.$$

На початку алгоритму задаємо точність $\varepsilon > 0$ (в коді задано $10^{(-6)}$). Початкове наближення задане в умові варіанту: $\beta_0 = (0.1, 11, 23)^T$

Розв'язуємо систему, яка є математичною моделлю коливання трьох мас. На першій ітерації використовуємо значення $\beta_0, y(t_0) = \bar{y}(t_0)$.

$$\begin{aligned}\frac{d^2 y_1(t)}{dt^2} + \frac{(c_2 + c_1)}{m_1} y_1(t) - \frac{c_2}{m_1} y_2(t) &= f_1(t), \\ \frac{d^2 y_2(t)}{dt^2} - \frac{c_2}{m_2} y_1(t) + \frac{(c_2 + c_3)}{m_2} y_2(t) - \frac{c_3}{m_2} y_3(t) &= f_2(t), \\ \frac{d^2 y_3(t)}{dt^2} - \frac{c_3}{m_3} y_2(t) + \frac{(c_4 + c_3)}{m_3} y_3(t) &= f_3(t).\end{aligned}$$

Тут $f_1(t), f_2(t), f_3(t)$ – зовнішні сили.

Цю систему диференціальних рівнянь потрібно звести до нормальної форми порядку 6. Це матиме вигляд

$$\frac{d}{dt} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{pmatrix} = \begin{pmatrix} \dot{y}_1 \\ \dot{y}_2 \\ \dot{y}_3 \\ \ddot{y}_1 \\ \ddot{y}_2 \\ \ddot{y}_3 \end{pmatrix} = \begin{pmatrix} x_4 \\ x_5 \\ x_6 \\ -\frac{(c_1+c_2)}{m_1}x_1 + \frac{c_2}{m_1}x_2 + f_1(t) \\ \frac{c_2}{m_2}x_1 - \frac{(c_2+c_3)}{m_2}x_2 + \frac{c_3}{m_2}x_3 + f_2(t) \\ \frac{c_3}{m_3}x_2 - \frac{(c_3+c_4)}{m_3}x_3 + f_3(t) \end{pmatrix}$$

де x - вектор стану $x = (y_1, y_2, y_3, y'_1, y'_2, y'_3)^T$

Для розв'язання цієї системи використовуємо метод Рунге-Кутти 4-го порядку.

$$\begin{aligned}\frac{dy}{dt} &= f(y, t), y(t_0) = y_0, \\ y_{n+1} &= y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4),\end{aligned}$$

де

$$\begin{aligned}k_1 &= hf(y_n, t_n), \\ k_2 &= hf\left(y_n + \frac{1}{2}k_1, t_n + \frac{1}{2}h\right), \\ k_3 &= hf\left(y_n + \frac{1}{2}k_2, t_n + \frac{1}{2}h\right), \\ k_4 &= hf(y_n + k_3, t_n + h), \\ t_{n+1} &= t_n + h.\end{aligned}$$

Далі ми визначаємо, наскільки сильно кожен параметр $\beta_k = \{c_1, m_1, m_2\}$ впливає на кожну змінну стану $y_i(t)$ системи в кожний момент часу.

Це досягається шляхом розв'язання системи диференціальних рівнянь, відомої як рівняння чутливості. Воно показує **часткові похідні** вектора стану за невідомими параметрами $\beta = (c_1, m_1, m_2)$ в кожний момент часу t . Значення $U(t)$ використовується для визначення, наскільки сильно кожен із невідомих параметрів впливає на кожну змінну стану, що дозволяє обчислити напрямок і величину корекції параметрів $\Delta\beta$.

$$\frac{dU(t)}{dt} = AU(t) + \frac{\partial(Ay)}{\partial\beta}, \quad U(t_0) = 0$$

Матриця A тут - матриця Якобі, має вигляд (показано нижче) і показує, як швидкість зміни кожної змінної стану (dt/dx_i) залежить від поточного значення кожної змінної стану (x_j).

$$A = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_6} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_6}{\partial x_1} & \cdots & \frac{\partial f_6}{\partial x_6} \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ -\frac{c_1+c_2}{m_1} & \frac{c_2}{m_1} & 0 & 0 & 0 & 0 \\ \frac{c_2}{m_2} & -\frac{c_2+c_3}{m_2} & \frac{c_3}{m_2} & 0 & 0 & 0 \\ 0 & \frac{c_3}{m_3} & -\frac{c_3+c_4}{m_3} & 0 & 0 & 0 \end{pmatrix}$$

Далі обраховуємо

$$\Delta\beta_k = \left(\int_{t_0}^T U^T(t, \beta_k) U(t, \beta_k) dt \right)^{-1} \int_{t_0}^T U^T(t, \beta_k) (\bar{y}(t) - y(t, \beta_k)) dt.$$

це вектор змін, який розраховується на кожній ітерації алгоритму. Він показує, наскільки і в якому напрямку потрібно зрушити кожен із невідомих параметрів. Обчислюємо наступне наближення

$$\beta_{k+1} = \beta_k + \Delta\beta_k.$$

Алгоритм закінчуємо, коли виконані одна з умов:

1. $\Delta\beta_{k+1} < \varepsilon$;
2. $I(\beta_{k+1}) < \varepsilon$.

Хід роботи

Визначаємо всі відомі та невідомі параметри, часовий інтервал і початкове наближення.

```
# ----- Параметри -----
P_SIZE = 7
C2_KNOWN, C3_KNOWN, C4_KNOWN, M3_KNOWN = 0.3, 0.2, 0.12, 18.0
BETA_INDICES = [3, 0, 1] # [c1, m1, m2]
BETA_0_GUESS = np.array([0.1, 11.0, 23.0])
M = len(BETA_INDICES)
TRUE_BETA_VALS = np.array([0.08, 10.0, 20.0])

T0, T, DT = 0.0, 50.0, 0.2
t_points = np.arange(T0, T + DT, DT)
X0 = np.zeros(6)
```

Математичну модель коливань зводимо до системи звичайних диференціальних рівнянь 1-го порядку (нормальна форма) розмірністю 6.

```
def system_ode(t, x, params): 2 usages
    m1, m2, m3, c1, c2, c3, c4 = params
    f1, f2, f3 = external_forces(t)

    dx1, dx2, dx3 = x[3], x[4], x[5]
    dx4 = -(c1 + c2)/m1 * x[0] + c2/m1 * x[1] + f1
    dx5 = c2/m2 * x[0] - (c2 + c3)/m2 * x[1] + c3/m2 * x[2] + f2
    dx6 = c3/m3 * x[1] - (c3 + c4)/m3 * x[2] + f3

    return np.array([dx1, dx2, dx3, dx4, dx5, dx6])
```

Для розв'язку рівняння чутливості $\frac{dU(t)}{dt} = AU(t) + \frac{\partial(Ay)}{\partial\beta}$, $U(t_0) = 0$

обчислюємо матрицю A та $\frac{\partial(Ay)}{\partial\beta}$

```
def sensitivity_ode_components(x, params): 1 usage
    m1, m2, m3, c1, c2, c3, c4 = params

    A = np.zeros((6, 6))
    A[0, 3], A[1, 4], A[2, 5] = 1.0, 1.0, 1.0
    A[3, 0] = -(c1 + c2)/m1; A[3, 1] = c2/m1
    A[4, 0] = c2/m2; A[4, 1] = -(c2 + c3)/m2; A[4, 2] = c3/m2
    A[5, 1] = c3/m3; A[5, 2] = -(c3 + c4)/m3

    dF_dBeta = np.zeros((6, M))
    dF_dBeta[3, 0] = -x[0]/m1
    dF_dBeta[3, 1] = (c1 + c2)/(m1**2)*x[0] - c2/(m1**2)*x[1]
    dF_dBeta[4, 2] = -c2/(m2**2)*x[0] + (c2 + c3)/(m2**2)*x[1] - c3/(m2**2)*x[2]

    return A, dF_dBeta
```

І розв'язуємо рівняння чутливості

```
def coupled_ode(t, X_coupled, full_params):
    x = X_coupled[:6]
    U_flat = X_coupled[6:]
    dx_dt = system_ode(t, x, full_params)
    A, dF_dBeta = sensitivity_ode_components(x, full_params)
    U = U_flat.reshape(6, M)
    dU_dt = A @ U + dF_dBeta
    return np.concatenate([dx_dt, dU_dt.flatten()])

# RK4
X_c0 = np.concatenate([x0, np.zeros(6*M)])
X_c_sol = np.zeros((len(t_points), len(X_c0)))
X_c_sol[0] = X_c0
for i in range(1, len(t_points)):
    X_c_sol[i] = rk4_step(coupled_ode, t_points[i-1], X_c_sol[i-1], dt, full_params_k)

y_t_beta_k = X_c_sol[:, :6]
U_flat_k_sol = X_c_sol[:, 6:]
```

Реалізовуємо метод Рунге-Кутти 4-го порядку для розв'язку системи рівнянь математичної моделі.

```
def rk4_step(f, t, x, dt, params): 3 usages
    k1 = f(t, x, params)
    k2 = f(t + dt/2, x + dt/2 * k1, params)
    k3 = f(t + dt/2, x + dt/2 * k2, params)
    k4 = f(t + dt, x + dt * k3, params)
    return x + dt/6 * (k1 + 2*k2 + 2*k3 + k4)

def rk4_integrate(f, x0, t_points, params): 1 usage
    X_sol = np.zeros((len(t_points), len(x0)))
    X_sol[0] = x0
    for i in range(1, len(t_points)):
        X_sol[i] = rk4_step(f, t_points[i-1], X_sol[i-1], t_points[i]-t_points[i-1], params)
    return X_sol
```

Далі обчислюємо $\Delta\beta_k$ та наступний крок β_{k+1}

```
E_t = y_bar_obs - y_t_beta_k
Integral_UT_U = np.zeros((M, M))
Integral_UT_E = np.zeros(M)

for i in range(len(t_points)):
    U_t = U_flat_k_sol[i].reshape(6, M)
    Integral_UT_U += U_t.T @ U_t * dt
    Integral_UT_E += U_t.T @ E_t[i] * dt

Delta_beta_k = inv(Integral_UT_U) @ Integral_UT_E
beta_next = beta_k + Delta_beta_k
I_beta_k = np.sum(E_t**2) * dt
```

Перевіряємо умови завершення алгоритму та за потреби продовжуємо його.

```
if np.linalg.norm(Delta_beta_k) < epsilon:
    # Оновлюємо beta_k до beta_next перед виходом
    beta_k = beta_next
    # Обчислюємо фінальне рішення з оновленими параметрами
    full_params_final = create_full_params(beta_k)
    X_c0_final = np.concatenate([x0, np.zeros(6*M)])
    X_c_sol_final = np.zeros((len(t_points), len(X_c0_final)))
    X_c_sol_final[0] = X_c0_final
    for i in range(1, len(t_points)):
        X_c_sol_final[i] = rk4_step(coupled_ode, t_points[i-1], X_c_sol_final[i-1],
                                   dt, full_params_final)
    y_t_beta_k = X_c_sol_final[:, :6]
    E_t_final = y_bar_obs - y_t_beta_k
    I_beta_k = np.sum(E_t_final**2) * dt
    break
beta_k = beta_next
```

Вивід програми:

```
Таблиця всіх ітерацій:
  k   c1_k   m1_k   m2_k   ||Delta_beta||   I(beta)
  1  0.100000 11.000000 23.000000   3.368135e+00  2.122048e+01
  2  0.077805  9.974657 19.791805   2.082844e-01  2.141974e-01
  3  0.079959  9.999009 19.998650   1.675126e-03  4.617772e-05
  4  0.080000 10.000000 20.000000   2.049463e-07  2.527672e-12

=====
ОТРИМАНІ ЗНАЧЕННЯ ПАРАМЕТРІВ:
=====
c1 = 0.080000000000
m1 = 10.0000000000
m2 = 20.0000000000
```

З графіку бачимо, що модель вийшла дуже близькою до початкового спостереження. Отже, код реалізований правильно і обчислення коректні.

